Assignment No : 03

Rollno : 13216

Problem Statement : Descriptive Statistics - Measures of Central Tendency and variability
Perform the following operations on any open source dataset (e.g., data.csv)

1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.) with numeric variables grouped by one of the qualitative (categorical) variable. For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups. Create a list that contains a numeric value for each response to the categorical variable.
2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Irisversicolor' of iris.csv dataset. Provide the codes with outputs and explain everything that you do in this step.

```
import pandas as pd
df = pd.read_csv("Mall_Customers.csv")
df.head()
```

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

```
df.mean(numeric_only=True)
```

```
CustomerID                100.50
Age                        38.85
Annual Income (k$)         60.56
Spending Score (1-100)     50.20
dtype: float64
```

```
df.median(numeric_only=True)
```

```
CustomerID                100.5
Age                        36.0
Annual Income (k$)         61.5
Spending Score (1-100)     50.0
dtype: float64
```

```
df.mode()
```

```
     CustomerID   Genre   Age  Annual Income (k$)  Spending Score (1-
100)
0             1  Female  32.0                54.0
42.0
```

```
1              2      NaN   NaN                   78.0
NaN
2              3      NaN   NaN                    NaN
NaN
3              4      NaN   NaN                    NaN
NaN
4              5      NaN   NaN                    NaN
NaN
..           ...      ...   ...                    ...
...
195          196      NaN   NaN                    NaN
NaN
196          197      NaN   NaN                    NaN
NaN
197          198      NaN   NaN                    NaN
NaN
198          199      NaN   NaN                    NaN
NaN
199          200      NaN   NaN                    NaN
NaN

[200 rows x 5 columns]
```

```
df.describe()
```

```
       CustomerID         Age  Annual Income (k$)  Spending Score (1-
100)
count  200.000000  200.000000          200.000000
200.000000
mean   100.500000   38.850000           60.560000
50.200000
std     57.879185   13.969007           26.264721
25.823522
min      1.000000   18.000000           15.000000
1.000000
25%     50.750000   28.750000           41.500000
34.750000
50%    100.500000   36.000000           61.500000
50.000000
75%    150.250000   49.000000           78.000000
73.000000
max    200.000000   70.000000          137.000000
99.000000
```

```
df.std(numeric_only=True)
```

```
CustomerID                57.879185
Age                       13.969007
Annual Income (k$)        26.264721
```

```
Spending Score (1-100)    25.823522
dtype: float64
```

```
df.min(numeric_only=True)
```

```
CustomerID                 1
Age                       18
Annual Income (k$)        15
Spending Score (1-100)     1
dtype: int64
```

```
df.groupby("Genre")["Age"].mean()
```

```
Genre
Female    38.098214
Male      39.806818
Name: Age, dtype: float64
```

```
df.describe()
```

```
       CustomerID         Age  Annual Income (k$)  Spending Score (1-
100)
count  200.000000  200.000000          200.000000
200.000000
mean   100.500000   38.850000           60.560000
50.200000
std     57.879185   13.969007           26.264721
25.823522
min      1.000000   18.000000           15.000000
1.000000
25%     50.750000   28.750000           41.500000
34.750000
50%    100.500000   36.000000           61.500000
50.000000
75%    150.250000   49.000000           78.000000
73.000000
max    200.000000   70.000000          137.000000
99.000000
```

```
df.loc[:,'Age'].mean()
```

```
np.float64(38.85)
```

```
df.mean(axis=1, numeric_only=True)[0:4]
```

```
0    18.50
1    29.75
2    11.25
3    30.00
dtype: float64
```

```
df.loc[:,'Age'].max(skipna = False)
```

```
np.int64(70)
```

```
df.std(numeric_only=True)
```

```
CustomerID                57.879185
Age                       13.969007
Annual Income (k$)        26.264721
Spending Score (1-100)    25.823522
dtype: float64
```

```
df.loc[:,'Age'].std()
```

```
13.969007331558883
```

```
df.std(axis=1,numeric_only=True)[0:4]
```

```
0    15.695010
1    35.074920
2     8.057088
3    32.300671
dtype: float64
```

```
df.groupby(['Genre'])['Age'].mean()
```

```
Genre
Female    38.098214
Male      39.806818
Name: Age, dtype: float64
```

```
df_u = df.rename(columns={'Annual Income (k$)': 'Income'},
inplace=False)
```

```
df_u.groupby(['Genre']).Income.mean()
```

```
Genre
Female    59.250000
Male      62.227273
Name: Income, dtype: float64
```

```
from sklearn import preprocessing
```

```
enc = preprocessing.OneHotEncoder()
```

```
enc_df = pd.DataFrame(enc.fit_transform(df[['Genre']]).toarray())
```

```
enc_df
```

```
       0    1
0    0.0  1.0
1    0.0  1.0
2    1.0  0.0
3    1.0  0.0
```

```
4    1.0  0.0
..   ...  ...
195  1.0  0.0
196  1.0  0.0
197  0.0  1.0
198  0.0  1.0
199  0.0  1.0

[200 rows x 2 columns]

df_encode = df.join(enc_df)

df_encode

     CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-
100)    0  \
0             1    Male   19                  15
39  0.0
1             2    Male   21                  15
81  0.0
2             3  Female   20                  16
6  1.0
3             4  Female   23                  16
77  1.0
4             5  Female   31                  17
40  1.0
..          ...     ...  ...                 ...                      .
..  ...
195         196  Female   35                 120
79  1.0
196         197  Female   45                 126
28  1.0
197         198    Male   32                 126
74  0.0
198         199    Male   32                 137
18  0.0
199         200    Male   30                 137
83  0.0

       1
0    1.0
1    1.0
2    0.0
3    0.0
4    0.0
..   ...
195  0.0
196  0.0
197  1.0
198  1.0
```

```
199  1.0

[200 rows x 7 columns]
```

```
df_encode.head()
```

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0  \
0           1    Male   19                  15                      39
0.0
1           2    Male   21                  15                      81
0.0
2           3  Female   20                  16                       6
1.0
3           4  Female   23                  16                      77
1.0
4           5  Female   31                  17                      40
1.0

     1
0  1.0
1  1.0
2  0.0
3  0.0
4  0.0
```

```
df.min(numeric_only=True)
df.max(numeric_only=True)
```

```
CustomerID                200
Age                        70
Annual Income (k$)        137
Spending Score (1-100)     99
dtype: int64
```